1 · Ambiente e estado markoviano

Aprender qual memória um LLM pode esquecer: despejo de KV-cache com orçamento fixo

O AMBIENTE

Um LLM lê um texto longo token a token e guarda as chaves e valores (K, V) de cada token no **KV-cache**. O cache comporta no máximo **B = 16 blocos** de 16 tokens. Quando chega um bloco novo com o cache cheio, o agente decide qual bloco sai.



KV-cache (16 blocos) novo

sink (início) · muito atendido · pouco atendido · recente

Episódio: um texto de ~1.000–2.000 tokens (64–128 blocos) → uma decisão por bloco novo após o cache encher. Dinâmica: o próprio texto e o padrão de atenção do modelo.

ESTADO s_t — O QUE O AGENTE OBSERVA

1. **Idade** há quantos passos o bloco entrou no cache
2. **Atenção acumulada** média da atenção recebida desde que entrou
3. **Atenção recente** média nos últimos k tokens (tendência)
4. **Sink / posição** se é bloco inicial e posição relativa no texto
5. **Global** progresso no texto (t / T)

É Markov? Quase. A atenção acumulada resume a história (estatística suficiente). Mas a importância futura de um bloco depende do texto que ainda não chegou → na prática, um **POMDP**.

Ihury Kewin · Willian Casa Grandi

2 · Ações do agente

A cada bloco novo com o cache cheio, escolher quem sai: $a_t \in \{0, 1, \ldots, B\}$

1 Cache cheio

Chega o bloco B+1. Há B+1 candidatos: os 16 do cache e o próprio bloco novo.

2 Avalia cada opção

Função ação-valor Q(s, a): retorno esperado de despejar o candidato a no estado s.

3 Despeja o de maior Q

Ação = argmax_a Q(s, a); o bloco escolhido sai e seus K, V somem para sempre.

EXEMPLO: Q(s, a) DE DESPEJAR CADA UM DOS 17 CANDIDATOS



Extensão: a_t = (bloco, operação), com operação ∈ {despejar, quantizar para 4 bits}: libera menos memória, mas perde menos informação.

Baselines a vencer: janela deslizante (sai o mais antigo), StreamingLLM (sinks + recentes), H2O (sai o menos atendido) e aleatório.

3 · Recompensas

Punir o quanto o modelo “se desvia” por ter esquecido, e premiar lembrar o que importa

DENSA — A CADA TOKEN

$$r_t = -\,KL(\,p_cheio \,\|\, p_reduzido\,)$$

Quanto a distribuição do próximo token, dado o texto até t, com o cache reduzido se afasta da distribuição com o cache completo. Zero se o despejo não mudou nada.

No ambiente de treino (proxy): r_t = − massa de atenção que os blocos já despejados receberiam nos próximos tokens.

TERMINAL — FIM DO TEXTO

+10

se o modelo recupera a “agulha”: uma senha escondida no início do texto e perguntada no final. 0 caso contrário.

Restrição dura

o cache nunca passa de B blocos, então não há penalidade de estouro.

Objetivo

maximizar $G = \Sigma\, \gamma^t\, r_t + R_final$, com $\gamma = 0{,}99$.

Próximos passos

ambiente proxy e agentes com o que veremos na disciplina; teste no GPT-2.